

Caiet de exerciții — Design Patterns în C#

Opt exerciții pe Strategy, Observer, Factory Method și Decorator. Primele șase sunt hint-free; ultimele au scheletul dat în repo.

Cum lucrezi cu caietul ăsta

Sunt 8 exerciții, gradate (`[usor]` , `[mediu]` , `[provocare]`). Nu sunt toate la fel — citește tabelul înainte să începi:

Partea	Exerciții	Cum lucrezi	Folderul
I — Strategy	S1, S2, S3	hint-free : primești problema, contractul și ce trebuie să iasă; clasele, contextul și testul le proiectezi tu	ți-l faci tu (<code>ex5</code> , <code>ex6</code> , ...)
II — Observer	O1, O2, O3	la fel	ți-l faci tu (<code>ex7</code> , <code>ex8</code> , <code>ex9</code>)
III — Factory Method	F1	ghidat : contractele, constructoarele și validările sunt scrise; tu completezi metodele rămase pe <code>NotImplementedException</code>	există deja : <code>ex10/</code>
IV — Decorator	D1	ghidat, la fel	există deja : <code>ex11/</code>
	D2	reparație în cod pe care l-ai scris tu	<code>ex1/</code> și <code>ex6/</code> , fișiere existente

Părțile I și II le-ai făcut deja (`ex5` – `ex9`) — au rămas aici ca referință și pentru recapitulare. Ce urmează e Partea III și IV.

La cele ghidate, cerința completă e în `exN/Cerinta.pdf`, lângă cod. Reguli valabile peste tot:

- Fără `is`, `as`, `GetType()`. Fără `switch` / `if` pe tip în context/subiect.
- Fără `List` / `Dictionary` / LINQ (excepție: `string.Join` unde ajută) — array-uri brute și `for`.
- Mesajele de eroare în engleză.
- Fiecare exercițiu are un `Testare` chemat din `Program.cs`, ca la celelalte.

Definiția de „gata” e la finalul fiecărui exercițiu: dacă output-ul arată acolo, ai terminat.

Teoria e în `TEORIE.pdf` — Strategy la lecția 1, Observer la lecția 2, Factory Method la lecția 3, Decorator la lecția 4. Recitește-o dacă te blochezi, dar încearcă întâi singur.

Partea I — Strategy

Reține forma: o familie de algoritmi interschimbabili în spatele unui contract; contextul ține unul și îl întreabă, fără să știe care e.

S1 — Validator de parolă [usor]

Context. Un câmp de parolă verifică dacă o parolă respectă o **politică**. Politica se poate schimba (site-uri diferite, cerințe diferite), fără să rescreești câmpul.

Contract.

```
public interface IPoliticaParola
{
    string Nume { get; }
    bool EsteValida(string parola);
}
```

Ce construiești.

Clasă	Regula
PoliticaSimpla	minim 6 caractere
PoliticaMedie	minim 8 caractere și conține cel puțin o cifră
PoliticaPuternica	minim 8 caractere, o cifră, o literă mare și un caracter care nu e literă/cifră
CampParola (Context)	ține o <code>IPoliticaParola</code> ; <code>Verifica(string parola)</code> întoarce <code>true/false</code> ; <code>SchimbaPolitica(...)</code>

Test. Aceeași parolă (ex: "abc123") verificată cu toate 3 politicile; apoi una puternică (ex: "Abc123!x"). Afișează numele politicii și rezultatul.

Gata când vezi aceeași parolă acceptată de politica simplă și respinsă de cea puternică, doar schimbând strategia — fără niciun `if` pe „ce fel de politică” în `CampParola` .

S2 — Comision de vânzare [mediu]

Context. La o vânzare se calculează comisionul agentului. Firma schimbă des modul de calcul; nu vrei să atingi clasa `Vanzare` de fiecare dată.

Contract.

```
public interface IComision
{
    string Nume { get; }
    decimal Calculeaza(decimal valoareVanzare);
}
```

Ce construiești.

Clasă	Formula
<code>ComisionFix</code>	o sumă fixă, dată în constructor (ex: 50), indiferent de valoare
<code>ComisionProcent</code>	un procent din valoare, dat în constructor (ex: 10%)
<code>ComisionPePraguri</code>	5% pentru partea până la 1000, 10% pentru cât depășește 1000
<code>Vanzare</code> (Context)	ține valoarea și o <code>IComision</code> ; <code>Comision()</code> întoarce suma; <code>SchimbaComision(...)</code>

Test. O vânzare de `1500` , calculată cu toate 3 strategiile. Verifică manual `ComisionPePraguri` :
 $5\% \cdot 1000 + 10\% \cdot 500 = 100$.

Gata când cele 3 strategii dau 3 valori diferite pe aceeași vânzare, iar `ComisionPePraguri` dă `100` la `1500` .

S3 — Reducere compusă [provocare]

Context. Extinzi S2. Vrei să aplici o reducere PESTE un comision deja calculat — și eventual încă una peste aceea.

Ce construiești. O strategie `ComisionCuPlafon` care primește în constructor **altă** `IComision` și o valoare maximă, și întoarce `min(comision_interior, plafon)`. Apoi o `ComisionCuBonus` care primește altă `IComision` și adaugă o sumă fixă.

Test. Compune-le: `ComisionCuPlafon(200, ComisionProcent(20%))` pe o vânzare de `1500` → `min(300, 200) = 200`. Apoi împachetează rezultatul într-un `ComisionCuBonus(50, ...)` → `250`.

Gata când poți înlănțui strategii care se împachetează una pe alta și rezultatul se calculează corect de la interior spre exterior.

Întrebare. O strategie care primește altă strategie de același tip și o „îmbunătățește”... îți sună cunoscut din `ex1` (bonusul `LivrareCuReducere`)? Țasta e un **Decorator** deghizat — îl vom numi așa la lecția 4.

Partea II — Observer

Reține forma: o sursă își schimbă starea și îi anunță pe toți abonații, fără să știe cine sunt sau ce fac. Nu așteaptă niciun răspuns înapoi.

01 — Canal cu abonați [usor]

Context. Un canal publică videoclipuri. Când apare unul nou, toți abonații sunt anunțați — fiecare în felul lui.

Contract.

```
public interface IAbonat
{
    void Notifica(string titluVideo);
}
```

Ce construiești.

Clasă	La <code>Notifica("X")</code>
<code>AbonatEmail</code>	<code>[EMAIL] Video nou: X</code>
<code>AbonatPush</code>	<code>[PUSH] X</code>
<code>Canal</code> (Subject)	ține <code>IAbonat[]</code> prin constructor; <code>PublicaVideo(string titlu)</code> anunță toți abonații

Bonus. `Aboneaza` / `Dezaboneaza` care cresc/micșorează array-ul manual (fără `List`). Atenție la dimensionarea array-ului la `Dezaboneaza` — numără întâi câți rămân, nu presupune că scoți exact unul. (Da, e capcana din review-ul tău la `ex2`.)

Gata când o publicare declanșează reacția tuturor abonaților, iar după un `Dezaboneaza` cel scos nu mai primește nimic.

O2 — Cont bancar [mediu]

Context. La fiecare tranzacție pe cont, mai mulți observatori reacționează — dar unul reacționează doar uneori.

Contract.

```
public interface IObservatorCont
{
    void Actualizeaza(decimal soldNou);
}
```

Ce construiești.

Clasă	Reacția
<code>NotificatorSms</code>	<code>[SMS] Sold nou: X</code> la fiecare schimbare
<code>JurnalAudit</code>	<code>[AUDIT] Sold inregistrat: X</code> la fiecare schimbare
<code>AlertaSoldMic</code>	afișează <code>[ALERTA] Sold sub prag: X</code> numai dacă <code>soldNou < prag</code> (pragul dat în constructor); altfel tace
<code>Cont</code> (Subject)	ține soldul și <code>IObservatorCont[]</code> ; <code>Depune(decimal)</code> și <code>Retrage(decimal)</code> schimbă soldul și anunță observatorii

Test. Pornești de la `1000`, faci `Retrage(950)` (sold 50 → alerta se declanșează), apoi `Depune(500)` (sold 550 → alerta tace). `Retrage` care ar duce soldul sub 0 aruncă `InvalidOperationException`.

Gata când SMS și audit reacționează la fiecare mișcare, dar alerta apare **doar** când soldul scade sub prag.

O3 — Licitație cu auto-licitator [provocare]

Context. La o licitație, fiecare ofertă nouă îi anunță pe toți participanții cu noul preț. Un participant special re-licitează automat dacă e depășit.

Contract.

```
public interface IParticipant
{
    void OfertaNoua(decimal pretCurent);
}
```

Ce construiești.

Clasă	Reacția
Spectator	[SPECTATOR] Pret curent: X
AutoLicitator	are un <code>pretMaxim</code> (din constructor); dacă <code>pretCurent < pretMaxim</code> , plasează o ofertă nouă de <code>pretCurent + 10</code> înapoi în licitație
Licitatie (Subject)	ține prețul curent și <code>IParticipant[]</code> ; <code>Liciteaza(decimal suma)</code> acceptă doar sume mai mari decât prețul curent (altfel <code>ArgumentException</code>), setează prețul și anunță toți participanții

Atenție la bucla de feedback. `AutoLicitator` cheamă `Liciteaza` din interiorul lui `OfertaNoua` — deci o ofertă declanșează alte oferte. Gândește-te: ce oprește lanțul? (răspuns: `pretMaxim` — când nimeni nu mai poate depăși, notificările nu mai produc oferte noi). Ai grijă ca licitatorul să nu se supraliciteze pe sine.

Gata când o singură `Liciteaza` de la un spectator declanșează o „bătălie” automată care se oprește singură când se atinge `pretMaxim`, și vezi în output cum urcă prețul pas cu pas.

Întrebare. La O2, observatorii doar *reacționează*. La O3, un observator *declanșează o nouă schimbare în subiect*. Ce risc apare când observatorii pot modifica sursa pe care o ascultă, și de ce trebuie o condiție de oprire?

Partea III — Factory Method

Reține forma: `new` e o decizie luată când scrii codul, nu când îl rulezi. Dacă vrei ca *clasa* să fie aleasă la rulare, muți `new` -ul într-un obiect care poate fi ales la rulare.

F1 — Încărcarea utilizatorilor [mediu]

Exercițiul ăsta e **ghidat**, nu hint-free: scheletul e în `ex10/`, cu contractele scrise și metodele pe `NotImplementedException`. Cerința completă e în `ex10/Cerinta.md`.

Pe scurt: linii de forma `STUDENT,Ana,Popescu,anul 2` și `PROFESOR,Ion,Ionescu,Matematica` trebuie să devină obiecte `Student` sau `Profesor`. `IncarcatorUtilizatori` nu are voie să conțină niciun `new Student`, niciun `switch` pe tip și niciun `as` — singura comparație permisă e între **două șiruri de date**, `fabrici[i].Tip == campuri[0]`.

Gata când adaugi un tip nou de utilizator (`ADMIN`) scriind doar o clasă-model și o fabrică, fără să modifichi nicio linie din încărcător.

Întrebare. De ce nu te salvează polimorfismul aici, deși `Descriere()` e virtuală? Răspunde în scris, două rânduri. (Ai lovit exact problema asta în `academy`, în `CreateUser`.)

Partea IV — Decorator

Reține forma: o clasă care semnează contractul **și** ține înăuntru un alt obiect cu același contract. Îl cheamă și îi ajustează rezultatul. Se stivuiesc, iar ordinea contează.

D1 — Notificări împachetate [mediu]

Și asta e ghidat: scheletul e în `ex11/`, cerința completă în `ex11/Cerinta.md`.

Pe scurt: `INotificator` cu `NotificatorEmail` și `NotificatorSms` (SMS-ul aruncă peste 160 de caractere). Peste ele scrii trei decoratori — `CuPrefix`, `CuTrunchiere`, `CuJurnal` — care se pot combina în orice ordine.

Gata când rulezi același mesaj lung prin cele două ordini, `CuJurnal(CuTrunchiere(160, sms))` și `CuTrunchiere(160, CuJurnal(sms))`, obții **rezultate diferite** și poți explica de ce.

D2 — Numele lanțului [usor, dar obligatoriu]

Nu e un exercițiu nou, e o reparație în cod pe care l-ai scris deja.

Deschide `ex1/Models/LivrareCuReducere.cs`, `ex6/Models/ComisionCuPlafon.cs` și `ex6/Models/ComisionCuBonus.cs`. Toate trei sunt decoratori — și toate trei își întorc un `Nume` fix:

```
public string Nume { get; } = "Cu bonus";
```

Un decorator nu are voie să facă asta: își pierde identitatea a ce împachetează. Compune-l:

```
public string Nume => strategie.Nume + " + bonus";
```

Gata când `Testare6` scrie `Procent + plafon + bonus: 250.00` în loc de `Cu bonus: 250.00`, iar din output poți citi tot lanțul fără să deschizi codul.

Întrebare. `ComisionCuPlafon` își validează `comisionMax` în constructor. `ComisionCuBonus` nu-și validează `valFixa`. Sunt clase-frați, scrise în aceeași ședință. De ce contează, și ce s-ar întâmpla cu un bonus negativ?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro